

BÁO CÁO BẢO VỆ BACKEND

LifeBalance - Spring Boot Microservices

Mục tiêu	Giải thích được tại sao backend được thiết kế như hiện tại và khi hệ thống chạy thì chuyện gì xảy ra.
Phạm vi	12 module trong lifebalance-backend: 4 module dùng chung/hạ tầng và 8 service nghiệp vụ.
Đối tượng đọc	Sinh viên chuẩn bị bảo vệ đồ án, người review backend, hội đồng kỹ thuật.
Công nghệ chính	Java 21, Spring Boot 3.5.x, Spring Cloud, Eureka, Gateway, PostgreSQL, Flyway, Keycloak/JWT, Docker Compose.
Ngày tạo	25/08/2026
File	E:\lifebalance-be\lifebalance-backend\Bao_cao_Bao_ve_Backend_LifeBalance.docx

Thông điệp cần bảo vệ

Đừng chỉ nói backend có những file nào. Cần nói được: vì sao tách service, request đi qua hệ thống như thế nào, dữ liệu được lưu và bảo vệ ra sao, lỗi/transaction/test/deployment được xử lý thế nào.

1. Ma trận đáp ứng yêu cầu báo cáo

Bảng này giúp đối chiếu nhanh từng yêu cầu bạn gửi với phần trả lời trong báo cáo.

Yêu cầu cần có	Đã đáp ứng ở phần
Tổng quan kiến trúc, 12 module, 4 module dùng chung, 8 service nghiệp vụ, lý do tách service	Phần 2, 3
Luồng vận hành request, ví dụ bấm Create Task	Phần 4
Controller, Service, Repository, Entity/Model, DTO, Config, Exception; vì sao không viết hết trong Controller	Phần 5
REST API, HTTP method, path variable, query parameter, request body, status code, pagination/filtering/sorting	Phần 6
Database, entity, bảng, quan hệ, PK/FK/index, xóa dữ liệu, database riêng hay chung	Phần 7
Authentication, Authorization, JWT, role/permission, token hết hạn, ownership	Phần 8
Gateway, Discovery Server, gọi trực tiếp service, gateway vs load balancer, discovery chết	Phần 9
Giao tiếp giữa service, REST/RestClient, đồng bộ/bất đồng bộ, service down	Phần 10
Validation, error handling, status 400/401/403/404/409/500	Phần 11
Transaction, commit/rollback, distributed transaction	Phần 12
Testing backend	Phần 13
Deployment, performance, hạn chế	Phần 14
Thứ tự học backend	Phần 15
Bảng 10 câu cho từng module	Phần 16

2. Tổng quan kiến trúc backend

Backend đang dùng kiến trúc multi-module Maven theo hướng microservices. Mỗi service nghiệp vụ là một Spring Boot application độc lập, có API riêng, database/migration riêng, và được gọi qua gateway. Các concern dùng chung được tách thành module thư viện hoặc module hạ tầng.

Câu trả lời ngắn khi hội đồng hỏi backend dùng kiến trúc gì

Backend dùng kiến trúc microservices trên nền Spring Boot multi-module Maven. Gateway là entry point,

Discovery Server quản lý service registry, các service nghiệp vụ xử lý bounded context riêng, còn common/security chuẩn hóa response, lỗi và bảo mật.

Nhóm	Module	Vai trò
4 module dùng chung/hạ tầng	discovery-server	Service registry: service đăng ký và tìm nhau qua service name.
4 module dùng chung/hạ tầng	gateway	API entry point: authentication/routing/CORS/logging ở biên hệ thống.
4 module dùng chung/hạ tầng	lifebalance-common	Thư viện chung: ApiResponse, ApiError, exception/common utilities.
4 module dùng chung/hạ tầng	lifebalance-security	Thư viện security: JWT resource server, Keycloak mapper, 401/403 handler.
8 service nghiệp vụ	identity-service	Danh tính, role, permission, audit, administration support.
8 service nghiệp vụ	task-service	Task, category, tag, reminder, recurring rule, task history.
8 service nghiệp vụ	timeline-service	Timeline, placement, day/week view, availability, conflict.
8 service nghiệp vụ	resource-capital-service	Vốn nguồn lực: time/money capital, cycle, adjustment, allocation.
8 service nghiệp vụ	finance-service	Account, category, budget, transaction, recurring transaction, finance report.
8 service nghiệp vụ	notification-service	Notification, template, preference, delivery, history.
8 service nghiệp vụ	analytics-service	Dashboard, tracking/evaluation, actual record, report, trend.
8 service nghiệp vụ	ai-service	Recommendation, insight, conversation, AI history.

3. Vì sao cần 8 service mà không viết chung một backend?

- Mỗi service đại diện một bounded context: identity, task, timeline, resource-capital, finance, notification, analytics, AI.
- Mỗi service có dữ liệu và rule nghiệp vụ khác nhau; tách ra giúp code dễ review, test và bảo trì.
- Các service có mức rủi ro khác nhau: identity cần bảo mật chặt, resource-capital có nhiều invariant, analytics thiên về đọc/tổng hợp.

- Khi scale, có thể tăng instance của service chịu tải cao như gateway/task/analytics mà không scale toàn bộ backend.
- Khi một service hỗ trợ như notification/AI lỗi, core service vẫn có thể tiếp tục nếu integration được thiết kế degrade gracefully.
- Nhược điểm là hệ thống phức tạp hơn: cần gateway, discovery, cấu hình mạng, observability và xử lý lỗi phân tán.

Câu trả lời mẫu

Em không viết chung một backend vì hệ thống có nhiều miền nghiệp vụ khác nhau. Nếu gom hết vào một monolith, controller/service/entity sẽ phụ thuộc chéo và khó test. Tách thành 8 service giúp mỗi module chịu trách nhiệm rõ ràng, có database/migration riêng, có thể scale và review độc lập. Em vẫn gom phần dùng chung như response, error và security vào common/security để tránh lặp code.

4. Luồng vận hành của một request

Ví dụ quan trọng nhất khi bảo vệ: người dùng bấm Create Task ở frontend.

1. Frontend gửi POST /api/tasks tới gateway kèm Authorization: Bearer <JWT> và request body chứa thông tin task.
2. Gateway nhận request ở port 8080, đi qua security filter để kiểm tra token có hợp lệ không.
3. Gateway nhìn route Path=/api/tasks/** và định tuyến tới lb://TASK-SERVICE.
4. Spring Cloud LoadBalancer hỏi Eureka để tìm instance task-service đang sống.
5. TaskController nhận POST /api/tasks, map request body vào DTO và validate dữ liệu cơ bản.
6. TaskController gọi TaskService, không tự xử lý business logic trong controller.
7. TaskService lấy userId/role từ security context, kiểm tra owner, kiểm tra rule như deadline, status, category/tag.
8. TaskService tạo Task entity, gọi TaskRepository để lưu xuống database PostgreSQL của task-service.
9. Trong cùng transaction nội bộ, service có thể ghi TaskChangeHistory hoặc dữ liệu liên quan.
10. Sau khi task thay đổi, task-service gọi integration bằng RestClient tới timeline-service, notification-service hoặc analytics-service nếu cấu hình bật.
11. Service trả DTO/ApiResponse về controller; gateway trả HTTP response về frontend.

Tầng	Trách nhiệm khi Create Task
Frontend	Thu thập input, gửi request đúng endpoint/body/token.
Gateway	Xác thực token ở biên, route tới task-service.
TaskController	Nhận HTTP request, validate DTO, gọi service.
TaskService	Business logic: owner, trạng thái, category/tag, reminder, history, integration.

TaskRepository	Lưu/đọc entity Task trong DB task-service.
Database	Lưu bảng tasks, task_tags, reminders/history tùy nghiệp vụ.
Integration	Sync timeline task, tạo notification, seed actual record nếu có.

5. Cách tổ chức code trong mỗi service

Layer/package	Công dụng	Điểm cần nói khi bảo vệ
Controller	Nhận HTTP request, đọc path/query/body, validate cơ bản, gọi Service, trả response.	Controller mỏng; không chứa nghiệp vụ phức tạp.
Service	Xử lý business logic, transaction boundary, gọi repository/integration.	Đây là nơi đặt rule chính và là phần cần review kỹ nhất.
Repository	Làm việc với database qua Spring Data JPA.	Chỉ truy vấn/persistence, không quyết định nghiệp vụ.
Entity/Model/Domain	Đại diện dữ liệu trong DB và quan hệ JPA.	Map bảng, PK/FK, enum, quan hệ 1-n/n-n.
DTO	Dữ liệu truyền vào/ra API.	Không expose entity trực tiếp; validate request rõ ràng.
Config	Cấu hình bean, security, integration, profile/env.	Không hard-code URL/secret trong code.
Exception/Error	Mã lỗi, exception, global exception handler.	Trả lỗi nhất quán: 400/401/403/404/409/500.
Integration/Infrastructure	Adapter gọi service ngoài hoặc hệ thống ngoài.	Tách khỏi domain để thay đổi REST/message/provider dễ hơn.

Tại sao không viết toàn bộ logic trong Controller?

Vì controller chỉ nên là lớp giao tiếp HTTP. Nếu đặt hết logic trong controller thì code khó test unit, khó tái sử dụng, khó quản lý transaction và dễ trộn validation, business rule, persistence, integration vào một nơi. Service layer giúp tách trách nhiệm và review nghiệp vụ rõ hơn.

6. API Design

Backend dùng REST API. Endpoint được đặt theo tài nguyên, dùng HTTP method để biểu diễn hành động.

Thành phần API	Ý nghĩa	Ví dụ trong hệ thống
GET	Đọc dữ liệu.	GET /api/tasks, GET /api/tasks/{id}, GET /api/timeline/week

POST	Tạo mới hoặc thực hiện action có side effect.	POST /api/tasks, POST /api/tasks/{id}/duplicate
PUT	Cập nhật toàn phần/tài nguyên ổn định.	PUT /api/categories/{id}, PUT /api/notifications/preferences
PATCH	Cập nhật một phần/trạng thái.	PATCH /api/tasks/{id}/complete, PATCH /api/v1/capital-allocations/{id}
DELETE	Xóa hoặc archive tùy rule.	DELETE /api/tasks/{id}, DELETE /api/roles/{id}
Path variable	Định danh tài nguyên trong URL.	{id}, {taskId}, {roleId}
Query parameter	Lọc, phân trang, chọn khoảng thời gian.	page, size, status, from, to, periodStart, periodEnd
Request body	Payload JSON khi tạo/cập nhật.	CreateTaskRequest, CreateNotificationRequest
Response DTO	Dữ liệu trả về đã được kiểm soát.	TaskResponse, CapitalSummaryResponseDTO
Pagination/filtering/sorting	Giúp danh sách lớn không trả toàn bộ.	Các API list theo page/size/status/date; sorting tùy module hỗ trợ.

Ví dụ POST /api/tasks

Endpoint nhận JSON mô tả task như title, description, priority, deadline, category/tag/reminder tùy DTO. Backend validate dữ liệu bắt buộc, kiểm tra user từ JWT, kiểm tra category/tag thuộc user, tạo Task entity, lưu DB, ghi history/integration nếu cần và trả TaskResponse kèm HTTP status thành công.

7. Database và dữ liệu

Hệ thống dùng PostgreSQL. Compose cấu hình DB/user/password riêng cho từng service: lifebalance_identity, lifebalance_task, lifebalance_timeline, lifebalance_resource_capital, lifebalance_finance, lifebalance_notification, lifebalance_analytics và lifebalance_ai. Mỗi service có Flyway migration riêng trong src/main/resources/db/migration.

Database riêng hay chung?

Theo cấu hình Docker Compose, mỗi service nghiệp vụ có database riêng trong cùng PostgreSQL server. Đây gần với microservice chuẩn hơn so với dùng chung một database/schema. Vẫn cần lưu ý: cùng một PostgreSQL instance vật lý nên chưa phải tách hạ tầng DB hoàn toàn.

Service	Entity/bảng chính	Quan hệ cần biết
identity	users, roles, permissions, user_roles, role_permissions, audit_logs, support_tickets	User n-n Role qua user_roles; Role n-n Permission qua role_permissions; support ticket có requester/assignee.

task	tasks, categories, tags, task_tags, task_reminders, task_recurring_rules, task_change_histories	Task n-1 Category; Task n-n Tag qua task_tags; Task 1-n Reminder/RecurringRule/History.
timeline	timeline_tasks, timeline_placements, timeline_histories	TimelinePlacement n-1 TimelineTask; history gắn placement/task.
resource-capital	capital_cycles, time_capitals, money_capitals, capital_adjustments, capital_allocations, capital_histories	CapitalCycle là gốc; cycle liên quan time/money capital, adjustment, allocation, history, release/reallocation.
finance	finance_accounts, finance_categories, finance_budgets, financial_transactions, recurring_transaction_rules, finance_histories	Transaction gắn account/category/budget theo rule; recurring rule sinh giao dịch định kỳ.
notification	notifications, notification_templates, notification_preferences, notification_delivery_attempts, notification_histories	Notification có template/preference/delivery attempts/history.
analytics	actual_records, analytics_reports, evaluation_results, analytics_histories	Dữ liệu tổng hợp theo user/reference/time period; thiên về read model/report.
ai	ai_recommendations, ai_insights, ai_conversations, ai_messages, ai_histories	Conversation 1-n Message; recommendation/insight/history gắn user/context.

- Primary key chủ yếu là UUID/id để định danh tài nguyên.
- Foreign key nằm trong cùng database của service; không nên tạo FK trực tiếp sang database service khác.
- Index được thêm qua migration cho các trường hay query như owner/user/status/date/reference.
- Khi xóa dữ liệu cần phân biệt hard delete và soft delete/archive. Với dữ liệu audit/history/finance/notification thường nên archive để giữ truy vết.
- Quan hệ n-n nên có bảng trung gian như user_roles, role_permissions, task_tags.

8. Authentication và Authorization

Khái niệm	Giải thích ngắn
Authentication	Xác định người dùng là ai. Ví dụ: login đúng tài khoản, token hợp lệ.
Authorization	Xác định người dùng được phép làm gì. Ví dụ: role ADMIN có permission quản trị.
JWT	Token client gửi trong Authorization header; backend validate issuer/audience/expiry/signature.
Role	Nhóm quyền theo vai trò, ví dụ USER/ADMIN/STAFF.
Permission	Quyền cụ thể, ví dụ task:read hoặc administration-dashboard:read.

12. User login qua identity/Keycloak.
13. Hệ thống kiểm tra tài khoản/mật khẩu, sinh JWT/access token.
14. Client lưu token và gửi Authorization: Bearer <token> trong mỗi request.

15. Gateway/service dùng lifebalance-security để validate token.
16. Backend map user/role từ JWT vào security context.
17. Controller/service kiểm tra permission/owner; hợp lệ thì xử lý, không hợp lệ trả 401/403.

Câu hỏi hội đồng	Câu trả lời nên dùng
Token hết hạn thì sao?	JWT hết hạn bị JwtTimestampValidator từ chối, backend trả 401; client phải login/refresh token.
Password có lưu plaintext không?	Không nên lưu plaintext. Password thuộc quản lý identity/Keycloak và phải được hash/salt.
JWT chứa gì?	Thông tin định danh như subject/user id, issuer, audience, expiry, scope/role/claim cần thiết.
Role và Permission khác nhau thế nào?	Role là nhóm/quyền theo vai trò; permission là hành động cụ thể. User có role, role có nhiều permission.
User sửa request để truy cập dữ liệu người khác thì sao?	Backend không tin frontend; service phải kiểm tra owner/userId từ token với dữ liệu trong DB.

9. Gateway và Discovery Server

Thành phần	Vai trò	Điểm cần bảo vệ
Gateway	Entry point, routing, security/CORS/logging/rate limiting nếu bổ sung.	Frontend gọi gateway thay vì gọi từng service.
Discovery Server	Registry service; service đăng ký và tìm nhau qua service name.	Giúp gateway dùng lb://TASK-SERVICE thay vì hard-code URL.
LoadBalancer	Chọn instance cụ thể khi có nhiều instance cùng service.	Gateway + discovery + load balancer phối hợp để route request.

Câu hỏi	Trả lời ngắn
Nếu biết URL Task Service thì gọi trực tiếp được không?	Về kỹ thuật có thể nếu network mở, nhưng thiết kế chuẩn là đi qua gateway để áp dụng auth/CORS/logging/routing nhất quán. Trong Docker, service nghiệp vụ nên ở internal network.
Gateway khác Load Balancer ở đâu?	Gateway hiểu API/path/security ở tầng ứng dụng; load balancer chủ yếu phân phối traffic tới instance. Gateway có thể dùng load balancer bên trong.
Discovery Server chết thì sao?	Instance đang chạy có thể còn hoạt động tạm theo cache, nhưng service mới/restart khó đăng ký và gateway khó tìm instance mới. Cần healthcheck/restart/HA nếu production.

10. Giao tiếp giữa các service

Trong code hiện tại, giao tiếp service-to-service đáng chú ý dùng REST đồng bộ qua Spring RestClient. Một số optional infrastructure như RabbitMQ/Redis có trong Docker profile, nhưng luồng chính hiện vẫn là REST.

Nguồn	Đích	Mục đích	Khi đích down
task-service	timeline-service	Sync task sang timeline task qua /api/timeline/tasks.	RestClientException được log warn; cần xem đây là eventual consistency risk.
task-service	notification-service	Tạo notification khi task thay đổi/reminder.	Core task có thể vẫn chạy, nhưng thông báo mất; nên nâng cấp bằng queue/outbox.
task-service	analytics-service	Seed actual record để analytics có dữ liệu đánh giá.	Log lỗi; analytics có thể thiếu record.
resource-capital-service	task-service	Kiểm tra allocation target task có thuộc current user không.	Nếu task data unavailable thì ném AllocationTargetUnavailableException, request allocation bị chặn để bảo toàn dữ liệu.
resource-capital-service	notification-service	Tạo cảnh báo capital.	Log warn; core capital có thể vẫn chạy nếu notification chỉ là side effect.

Câu hỏi khó: Task Service gọi Notification Service nhưng Notification Service bị down thì sao?

Với thiết kế hiện tại, notification là side effect nên task-service có thể log lỗi và không làm hỏng nghiệp vụ task chính. Tuy nhiên đây là hạn chế của REST đồng bộ: dễ mất notification nếu không có retry bền vững. Hướng tốt hơn là dùng message queue/outbox, retry, idempotency và circuit breaker.

11. Validation và Error Handling

Tình huống	HTTP status nên trả	Ý nghĩa
Request sai dữ liệu/thiếu field	400 Bad Request	DTO validation fail.
Chưa đăng nhập/không có token/token hết hạn	401 Unauthorized	Authentication fail.
Có token nhưng không có quyền	403 Forbidden	Authorization fail.
Không tìm thấy resource	404 Not Found	Id không tồn tại hoặc không thuộc user.

Conflict trạng thái/dữ liệu	409 Conflict	Ví dụ duplicate, over-allocation, rule lifecycle không hợp lệ.
Lỗi hệ thống/database/service ngoài	500 hoặc 503	Server error hoặc dependency unavailable.

- Global exception handler nên chuyển exception thành ApiError/ApiResponse thống nhất.
- Validation cơ bản đặt ở DTO/controller; validation nghiệp vụ đặt ở service/domain.
- Không trả stack trace hoặc thông tin nhạy cảm ra client.

12. Transaction và tính toàn vẹn dữ liệu

@Transactional được dùng nhiều ở service layer. Với một database cục bộ của service, transaction đảm bảo hoặc commit toàn bộ, hoặc rollback khi có exception.

Trường hợp	Cách hiểu khi bảo vệ
Tạo Task + ghi History trong cùng task-service	Có thể nằm trong một transaction local; nếu ghi history lỗi thì rollback cả request nếu exception không bị swallow.
Task Service gọi Timeline/Notification/Analytics	Đây là nhiều service khác database; không còn là transaction ACID đơn giản.
Bước thứ ba ở service khác lỗi	Phải chọn chiến lược: cho core transaction tiếp tục và ghi log/retry, hoặc fail request nếu side effect là bắt buộc.
Hướng nâng cấp	Outbox pattern, message queue, retry, idempotency, saga/compensation, circuit breaker.

13. Testing Backend

Module	Quy mô test hiện tại
discovery-server	1 Java / 2 test
gateway	1 Java / 3 test
lifebalance-common	10 Java / 2 test
lifebalance-security	13 Java / 9 test
identity-service	179 Java / 43 test
task-service	89 Java / 19 test
timeline-service	40 Java / 4 test
resource-capital-service	145 Java / 47 test

finance-service	74 Java / 5 test
notification-service	57 Java / 4 test
analytics-service	61 Java / 8 test
ai-service	55 Java / 5 test

- Không nên trả lời “em chạy thử thấy được”. Hãy nói: em test API theo case thành công, dữ liệu không hợp lệ, không có quyền, resource không tồn tại và conflict.
- Unit test phù hợp cho service/domain rule.
- Integration test phù hợp cho repository, migration, transaction và API flow.
- Postman/Swagger phù hợp để demo thủ công trước hội đồng.
- Security test cần kiểm tra 401/403, role/permission và ownership.

14. Deployment, performance và hạn chế

Chủ đề	Nội dung cần biết
Chạy hệ thống	Copy-Item .env.example .env; docker compose up -d --build.
Port chính	Gateway 8080, Eureka 8761, Keycloak 8082, PostgreSQL 5432.
Port debug service	identity 8091, task 8092, finance 8093, notification 8094, ai 8095, timeline 8096, analytics 8097.
Environment variables	DB URL/user/password, Keycloak issuer/client, Eureka URL, integration base URL, internal secret.
Logging/monitoring	Actuator health/readiness/liveness, Prometheus/Grafana khi bật monitoring profile.
Cache/message/storage	Redis/RabbitMQ/MinIO là optional profile, chưa phải luồng bắt buộc cho core business.
Scalability	Có thể scale service độc lập, nhưng cần kiểm soát DB connection, discovery, gateway và observability.

Hạn chế hiện tại	Tác động	Hướng cải thiện
Một số integration REST đồng bộ	Service đích down có thể mất side effect hoặc fail request.	Message queue/outbox/retry/circuit breaker.
Chưa có distributed tracing đầy đủ	Khó lần theo request qua nhiều service.	OpenTelemetry/Zipkin/Tempo.
Chưa benchmark tải lớn	Chưa chứng minh throughput/latency.	Load test bằng k6/JMeter/Gatling.
Cùng một PostgreSQL instance vật lý	Chưa tách hạ tầng DB hoàn toàn.	Tách instance/cluster khi production.
AI phụ thuộc provider ngoài nếu tích hợp thật	Rủi ro latency, chi phí, privacy.	Adapter, fallback, rate limit, prompt/data governance.

15. Thứ tự học backend để bảo vệ

18. Tổng thể kiến trúc.
19. Một request đi từ Gateway đến DB như thế nào.
20. Controller -> Service -> Repository.
21. Database: entity, bảng, quan hệ, PK/FK/index.
22. Security/JWT: authentication, authorization, role/permission, ownership.
23. Gateway + Discovery.
24. Service-to-Service.
25. Exception + Validation + Transaction.
26. Đi sâu từng module.
27. Testing + Deployment + Performance.

16. Bảng 10 câu phải biết cho từng module

Nếu trả lời được 10 câu trong bảng của từng module, bạn đã làm chủ backend ở mức đủ tốt để bảo vệ.

16. discovery-server

Module làm gì?	Quản lý service nào đang hoạt động để hệ thống không hard-code địa chỉ service.
Tại sao cần module này?	Cần cho kiến trúc microservice vì container/port có thể thay đổi.
API chính?	/eureka, /actuator/health, /actuator/prometheus
Entity chính?	Không có entity nghiệp vụ
Service gọi ai?	Spring Cloud Netflix Eureka Server, Actuator
 Ai gọi service này?	Gateway và các service khi đăng ký/tìm service
Security?	Endpoint vận hành public theo cấu hình; không chứa dữ liệu người dùng
Business logic khó nhất?	Đăng ký instance, heartbeat, trả danh sách service runtime
Lỗi nào có thể xảy ra?	Validation lỗi, không có quyền, resource không tồn tại, conflict trạng thái, database/dependency lỗi.
Nếu module chết?	Service đang chạy vẫn có thể hoạt động tạm bằng cache, nhưng service mới/restart khó đăng ký; gateway khó resolve instance mới

16. gateway

Module làm gì?	Cửa vào duy nhất của backend, che giấu địa chỉ thật của service và gom cross-cutting concern ở biên.
-----------------------	--

Tại sao cần module này?	Giúp frontend không phải biết từng service/port, để kiểm soát CORS, logging, auth và routing.
API chính?	/api/identity/**, /api/tasks/**, /api/v1/capital/**, /api/timeline/**, /api/finance/**, /api/notifications/**, /api/analytics/**, /api/ai/**
Entity chính?	Không có database/entity
Service gọi ai?	Eureka, Spring Cloud Gateway MVC, lifebalance-security
Ai gọi service này?	Frontend/mobile/client
Security?	Entry point kiểm tra JWT/security filter và route request tới service
Business logic khó nhất?	Route theo path, dùng lb://SERVICE-NAME qua Eureka
Lỗi nào có thể xảy ra?	Validation lỗi, không có quyền, resource không tồn tại, conflict trạng thái, database/dependency lỗi.
Nếu module chết?	Frontend không gọi được backend qua endpoint chính; service nội bộ vẫn có thể còn sống

16. lifebalance-common

Module làm gì?	Đảm bảo các service trả response/lỗi thống nhất.
Tại sao cần module này?	Tránh mỗi service định nghĩa format API khác nhau.
API chính?	Không expose endpoint
Entity chính?	Không có entity riêng
Service gọi ai?	Spring/Jackson/validation support
Ai gọi service này?	Tất cả service dùng ApiResponse, ApiError, exception/common utilities
Security?	Không trực tiếp xử lý security
Business logic khó nhất?	Chuẩn hóa response/error contract
Lỗi nào có thể xảy ra?	Validation lỗi, không có quyền, resource không tồn tại, conflict trạng thái, database/dependency lỗi.
Nếu module chết?	Không deploy độc lập; nếu contract thay đổi sai sẽ ảnh hưởng nhiều service khi build/runtime

16. lifebalance-security

Module làm gì?	Đóng gói authentication/authorization dùng chung.
Tại sao cần module này?	Security là concern xuyên suốt, cần thống nhất thay vì cấu hình rải rác.

API chính?	Không expose endpoint riêng
Entity chính?	Không có entity riêng
Service gọi ai?	Spring Security, OAuth2 Resource Server, JWT, Keycloak
Ai gọi service này?	Gateway và các service nghiệp vụ
Security?	Validate issuer/audience/token, map user/role, trả lỗi 401/403 chuẩn
Business logic khó nhất?	SecurityFilterChain stateless, public health/prometheus/swagger, còn lại authenticated
Lỗi nào có thể xảy ra?	Validation lỗi, không có quyền, resource không tồn tại, conflict trạng thái, database/dependency lỗi.
Nếu module chết?	Sai cấu hình có thể khiến endpoint bị mở nhầm hoặc bị 401/403 sai

16. identity-service

Module làm gì?	Quản lý danh tính, hồ sơ, vai trò, quyền và nghiệp vụ quản trị.
Tại sao cần module này?	Identity là vùng rủi ro bảo mật cao nên cần tách để quản trị và kiểm thử độc lập.
API chính?	/auth, /api/auth, /users, /api/users, /roles, /permissions, /audit-logs, /administration-support
Entity chính?	User, Role, Permission, UserRole, RolePermission, AuditLog, ActivityLog, SupportTicket, SystemAnnouncement, SystemConfiguration
Service gọi ai?	PostgreSQL, Flyway, Keycloak/admin integration, Eureka, security/common
Ai gọi service này?	Gateway, frontend/admin UI; các service tin JWT/role sinh từ identity/Keycloak
Security?	Quản lý role/permission, audit, admin operations; nhiều endpoint dùng @PreAuthorize
Business logic khó nhất?	Đồng bộ user/role/permission, phân quyền quản trị, audit log, support ticket
Lỗi nào có thể xảy ra?	Validation lỗi, không có quyền, resource không tồn tại, conflict trạng thái, database/dependency lỗi.
Nếu module chết?	Login/admin/role-permission bị ảnh hưởng; token đã cấp có thể còn dùng tới khi hết hạn

16. task-service

Module làm gì?	Quản lý công việc và kế hoạch hành động của người dùng.
Tại sao cần module này?	Task là core domain lớn, nhiều rule trạng thái và tích hợp nên cần tách riêng.
API chính?	/api/tasks, /api/categories, /api/tags, /api/tasks/reminders, /api/tasks/recurring-rules, /api/tasks/timeline

Entity chính?	Task, Category, Tag, TaskTag, TaskReminder, TaskRecurringRule, TaskChangeHistory, TimelinePlacement
Service gọi ai?	PostgreSQL, Flyway, Eureka, security/common; gọi timeline, notification, analytics bằng RestClient
Ai gọi service này?	Gateway/frontend; resource-capital kiểm tra task ownership
Security?	Request phải có JWT; service phải kiểm tra owner/userId khi thao tác task
Business logic khó nhất?	Vòng đời task, duplicate, plan/progress/complete/cancel, reminder, recurring rule, đồng bộ timeline/analytics/notification
Lỗi nào có thể xảy ra?	Validation lỗi, không có quyền, resource không tồn tại, conflict trạng thái, database/dependency lỗi.
Nếu module chết?	Người dùng không tạo/sửa task; timeline/analytics/notification mất nguồn dữ liệu chính

16. timeline-service

Module làm gì?	Biến task thành lịch thực thi theo ngày/tuần và quản lý xung đột thời gian.
Tại sao cần module này?	Scheduling có logic thời gian riêng, tách khỏi task để không làm task-service phình to.
API chính?	/api/timeline, /api/timeline/placements, /api/timeline/day, /api/timeline/week, /api/timeline/availability, /api/timeline/tasks
Entity chính?	TimelineTask, TimelinePlacement, TimelineHistory
Service gọi ai?	PostgreSQL, Flyway, Eureka, security/common
Ai gọi service này?	Gateway/frontend và task-service khi sync task
Security?	JWT/owner context; chỉ thao tác timeline của người dùng hiện tại
Business logic khó nhất?	Placement, reschedule/move/cancel, kiểm tra availability/conflict theo ngày/tuần
Lỗi nào có thể xảy ra?	Validation lỗi, không có quyền, resource không tồn tại, conflict trạng thái, database/dependency lỗi.
Nếu module chết?	Task vẫn có thể tồn tại nhưng không lập lịch/xem timeline được

16. resource-capital-service

Module làm gì?	Quản lý năng lượng/thời gian/tiền như nguồn lực cần phân bổ.
Tại sao cần module này?	Đây là domain đặc thù của LifeBalance, nhiều invariant nên tách riêng và test nhiều.
API chính?	/api/v1/capital, /api/v1/capital-cycles, /api/v1/capital-adjustments, /api/v1/capital-allocations

Entity chính?	CapitalCycle, TimeCapital, MoneyCapital, CapitalAdjustment, CapitalAllocation, CapitalHistory, CapitalReallocation, CapitalRelease
Service gọi ai?	PostgreSQL, Flyway, Eureka, security/common; gọi task-service kiểm tra ownership và notification-service để cảnh báo
Ai gọi service này?	Gateway/frontend; analytics/AI có thể dùng dữ liệu vốn nguồn lực
Security?	JWT/ownerId; validate allocation target thuộc user hiện tại
Business logic khó nhất?	Chu kỳ vốn, setup time/money capital, adjustment, allocation, reallocation, release, over-allocation policy
Lỗi nào có thể xảy ra?	Validation lỗi, không có quyền, resource không tồn tại, conflict trạng thái, database/dependency lỗi.
Nếu module chết?	Không theo dõi/phân bổ nguồn lực; task vẫn chạy nhưng mất góc nhìn capacity

16. finance-service

Module làm gì?	Quản lý tài chính cá nhân trong bức tranh cân bằng cuộc sống.
Tại sao cần module này?	Dữ liệu tài chính nhạy cảm và có rule tiền tệ riêng, không nên trộn với task/timeline.
API chính?	/api/finance/accounts, /api/finance/categories, /api/budgets, /api/transactions, /api/finance/reports, /api/finance/recurring-transactions
Entity chính?	FinanceAccount, FinanceCategory, FinanceBudget, FinancialTransaction, RecurringTransactionRule, FinanceHistory
Service gọi ai?	PostgreSQL, Flyway, Eureka, security/common
Ai gọi service này?	Gateway/frontend; analytics/AI có thể tổng hợp báo cáo
Security?	Dữ liệu tài chính phải gắn owner/user và không cho user truy cập dữ liệu người khác
Business logic khó nhất?	Account/category/budget, transaction, void/archive, recurring transaction, summary report
Lỗi nào có thể xảy ra?	Validation lỗi, không có quyền, resource không tồn tại, conflict trạng thái, database/dependency lỗi.
Nếu module chết?	Không ghi nhận/tổng hợp tài chính; các service khác ít bị chặn trực tiếp

16. notification-service

Module làm gì?	Tách logic thông báo khỏi service nghiệp vụ.
Tại sao cần module này?	Thông báo có retry/delivery/preference riêng, nên tách để tránh làm task/finance bị rối.
API chính?	/api/notifications, /api/notifications/templates, /api/notifications/preferences, /api/notifications/delivery, /api/notifications/history

Entity chính?	NotificationRecord, NotificationTemplate, NotificationPreference, NotificationDeliveryAttempt, NotificationHistory
Service gọi ai?	PostgreSQL, Flyway, Eureka, security/common; optional RabbitMQ/Redis theo profile
Ai gọi service này?	Gateway/frontend, task-service, resource-capital-service
Security?	JWT/owner; preference của user quyết định kênh/thông báo được gửi
Business logic khó nhất?	Template, preference, delivery state, read/unread/archive, retry delivery
Lỗi nào có thể xảy ra?	Validation lỗi, không có quyền, resource không tồn tại, conflict trạng thái, database/dependency lỗi.
Nếu module chết?	Core task/capital vẫn có thể chạy nhưng user không nhận nhắc việc/cảnh báo

16. analytics-service

Module làm gì?	Biến dữ liệu vận hành thành dashboard, đánh giá và báo cáo.
Tại sao cần module này?	Analytics là read/aggregate concern, nên tách khỏi service tạo dữ liệu.
API chính?	/api/analytics/dashboard, /api/analytics/reports, /api/analytics/tracking-evaluation, /api/analytics/evaluations, /api/analytics/actual-records
Entity chính?	ActualRecord, AnalyticsReport, EvaluationResult, AnalyticsHistory
Service gọi ai?	PostgreSQL, Flyway, Eureka, security/common
Ai gọi service này?	Gateway/frontend; task-service seed actual records
Security?	JWT/owner; report chỉ trên dữ liệu thuộc user
Business logic khó nhất?	Dashboard, planned-vs-actual, resource utilization, trends, compare periods, export report
Lỗi nào có thể xảy ra?	Validation lỗi, không có quyền, resource không tồn tại, conflict trạng thái, database/dependency lỗi.
Nếu module chết?	Người dùng vẫn tạo task/tài chính được nhưng mất báo cáo/đánh giá

16. ai-service

Module làm gì?	Đưa ra recommendation/insight hỗ trợ người dùng ra quyết định.
Tại sao cần module này?	AI thay đổi nhanh, có thể đổi provider/model nên cần tách khỏi core service.
API chính?	/api/ai/recommendations, /api/ai/insights, /api/ai/conversations, /api/ai/history
Entity chính?	AiRecommendation, AiInsight, AiConversation, AiMessage, AiHistory
Service gọi ai?	PostgreSQL, Flyway, Eureka, security/common; có thể thêm external AI provider

Ai gọi service này?	Gateway/frontend; dùng dữ liệu hệ thống để gợi ý
Security?	JWT/owner; cần kiểm soát dữ liệu đưa vào prompt/context
Business logic khó nhất?	Generate/apply/dismiss recommendation, insight, conversation/message history
Lỗi nào có thể xảy ra?	Validation lỗi, không có quyền, resource không tồn tại, conflict trạng thái, database/dependency lỗi.
Nếu module chết?	Mất tính năng gợi ý/insight; core task/finance/timeline vẫn hoạt động

17. Bộ câu trả lời nhanh khi bảo vệ

Câu hỏi	Trả lời ngắn gọn
Backend có bao nhiêu module?	12 module: 4 dùng chung/hạ tầng và 8 service nghiệp vụ.
4 module dùng chung là gì?	discovery-server, gateway, lifebalance-common, lifebalance-security.
8 service nghiệp vụ là gì?	identity, task, timeline, resource-capital, finance, notification, analytics, ai.
Vì sao tách microservice?	Tách bounded context, giảm coupling, test/scale/deploy độc lập, giữ service đúng trách nhiệm.
Bấm Create Task thì backend làm gì?	Frontend -> Gateway -> auth -> route task-service -> controller -> service -> repository -> DB -> integration -> response.
JWT được kiểm ở đâu?	lifebalance-security cấu hình OAuth2 Resource Server tại gateway/service, validate issuer/audience/expiry/signature.
Gateway khác discovery thế nào?	Gateway nhận request và route; discovery là danh bạ service instance.
Transaction nhiều service có rollback hết không?	Không đơn giản như một DB. Cần saga/outbox/compensation nếu cần nhất quán phân tán.

18. Nguồn tham chiếu trong repo

- pom.xml parent: danh sách module, Java/Spring Boot/Spring Cloud.
- gateway/src/main/resources/application.yaml: route lb://SERVICE-NAME.
- compose.yaml và .env.example: database riêng theo service, port, env, Keycloak/Eureka.
- lifebalance-security/.../LifebalanceSecurityAutoConfiguration.java: security filter, JWT, public/protected endpoints.
- task-service/.../RestTaskIntegrationClient.java: task-service gọi timeline/notification/analytics bằng RestClient.
- resource-capital-service/.../BasicAllocationTargetValidator.java: resource-capital gọi task-service kiểm tra ownership.

- `src/main/resources/db/migration/postgresql`: schema/table/index/constraint theo service.
- `src/main/java/**/controller, service, repository, model/domain, dto, error/exception`: cấu trúc code theo layer.